

SmartFinance: Automatizando a Soma de Despesas

Pratique! Dominando Listas, Loops e a Visão de Raio-X do VS Code



[15.50, 4.90, 3.60, 9.00]

0 Desafio SmartFinance

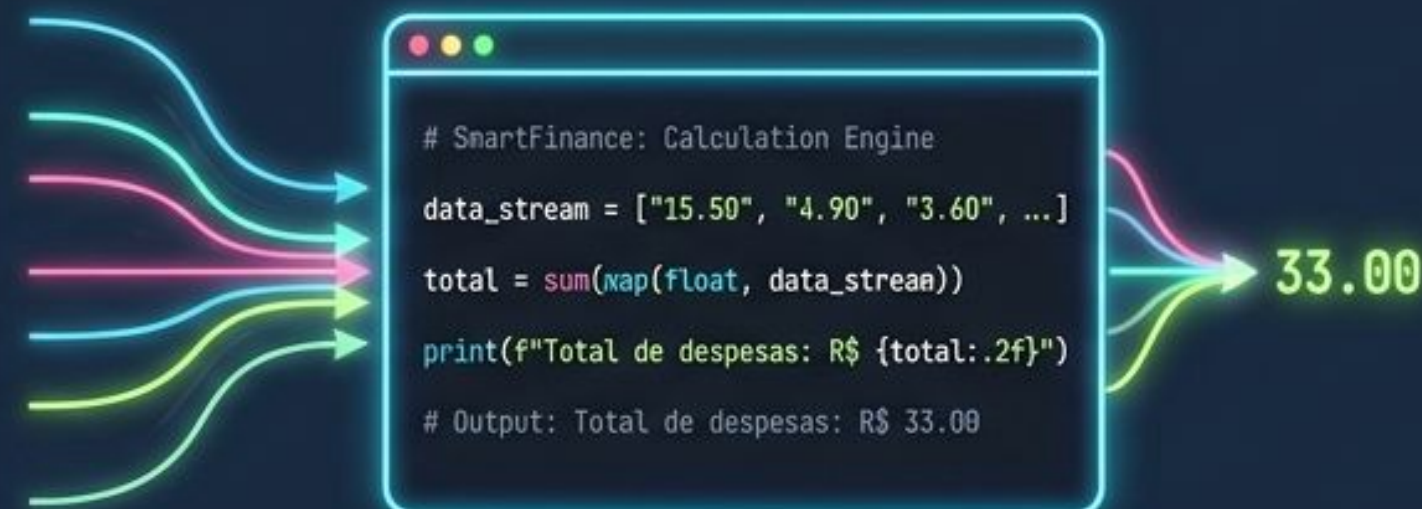
Calculadora vs. Código

A Abordagem Manual (0 Passado)



Lento, sujeito a erros humanos e impossível de escalar para milhares de transações. I

A Abordagem SmartFinance (0 Futuro)



Um script Python dinâmico que lê um histórico infinito de despesas e calcula o total instantaneamente. _

0 Mantra Infosec: PRATIQUE!

0 consenso entre programadores é que só aprendemos a programar a partir da prática! Abra o seu VS Code agora. Você não vai apenas ler este código, você vai construí-lo e inspecioná-lo de perto.

A Lógica Visual: Listas e Loops

A Arquitetura da Esteira



O Armazém (A Lista)

Conceito: Armazenamento sequencial de dados.

O Motor (O Loop "for")

Conceito: Iteração automática sobre a sequência.

O Acumulador (A Variável "total")

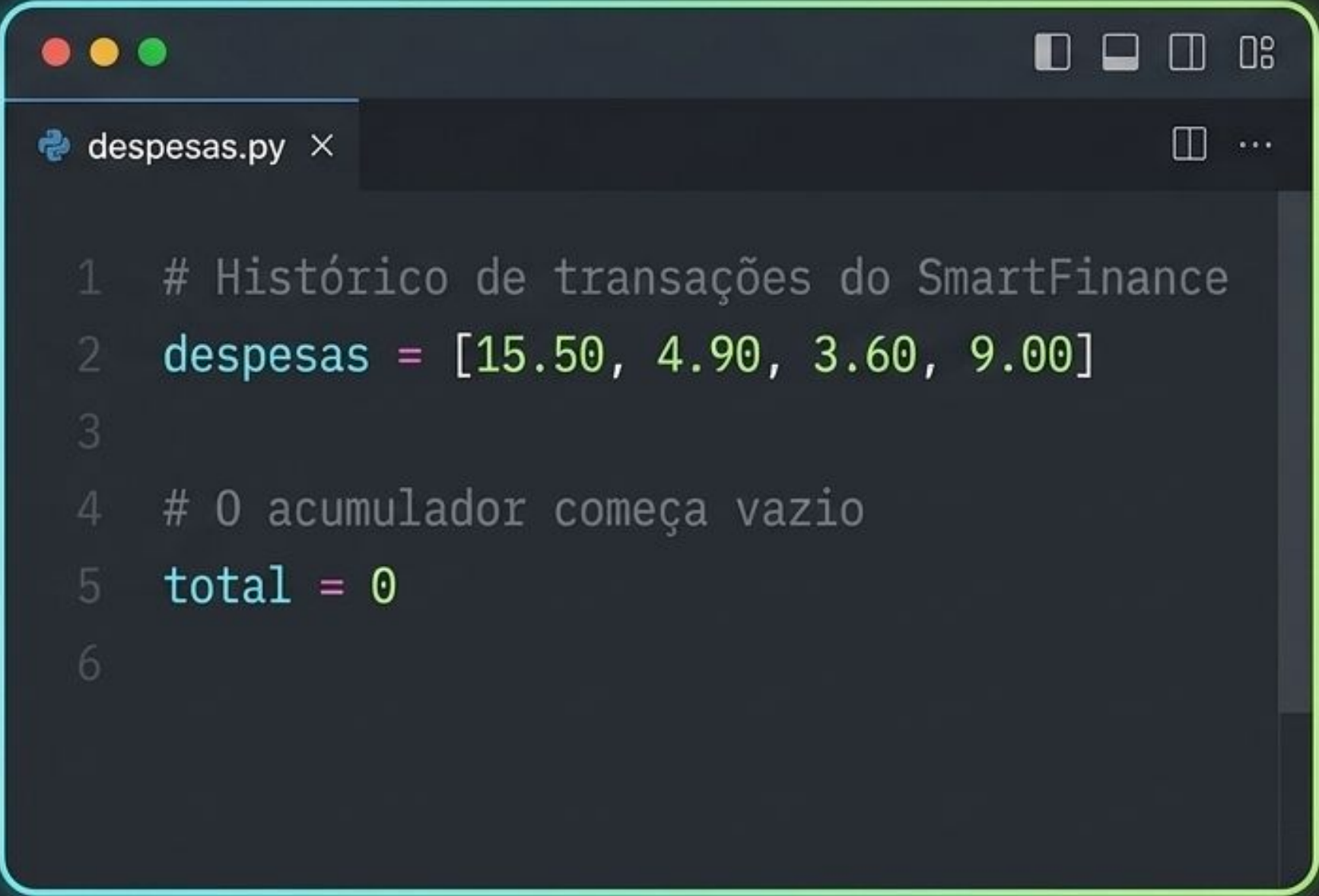
Conceito: Lógica de transação e controle de fluxo.

Passo 1: Preparando o Terreno

1. Crie um novo arquivo no VS Code chamado `despesas.py`.

2. Vamos declarar nossa lista de valores (usando colchetes []).

3. Vamos inicializar a variável `total` com o valor 0.



The screenshot shows a VS Code editor window with a single file named `despesas.py`. The code is as follows:

```
1  # Histórico de transações do SmartFinance
2  despesas = [15.50, 4.90, 3.60, 9.00]
3
4  # O acumulador começa vazio
5  total = 0
6
```


Passo 2: Construindo o Motor do Loop

Palavra-chave que inicia a repetição.

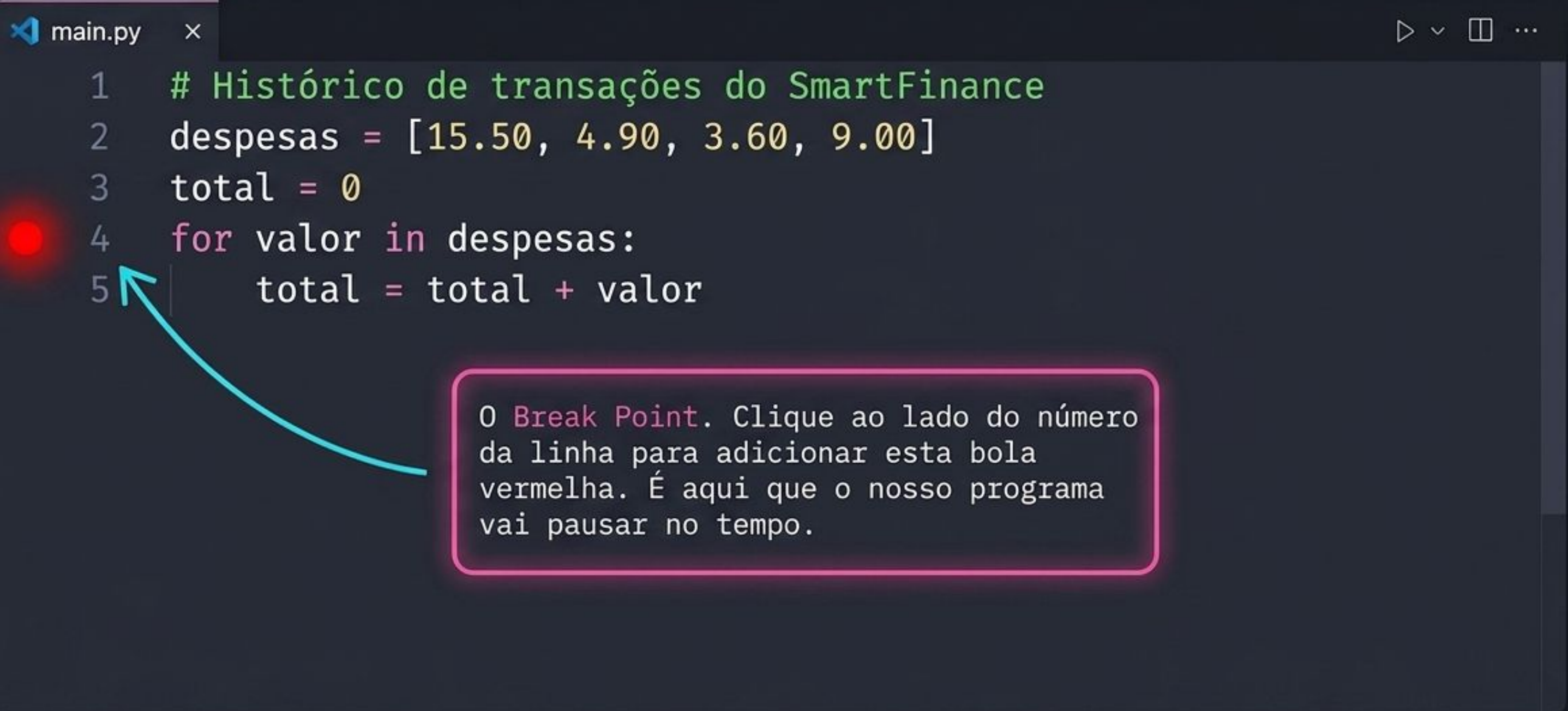
Variável temporária. A cada volta do loop, ela assume o valor de um item da lista.

Atenção: O símbolo de dois-pontos é obrigatório!

```
for valor in despesas:  
    total = total + valor
```

Indentação (Tab): Tudo que tem espaço à esquerda pertence ao bloco do loop.

Passo 3: Visão de Raio-X (0 Break Point)



The image shows a code editor window titled 'main.py'. The code is as follows:

```
1  # Histórico de transações do SmartFinance
2  despesas = [15.50, 4.90, 3.60, 9.00]
3  total = 0
4  for valor in despesas:
5      total = total + valor
```

A red dot is placed to the left of line 5. A blue arrow points from a text box to this dot.

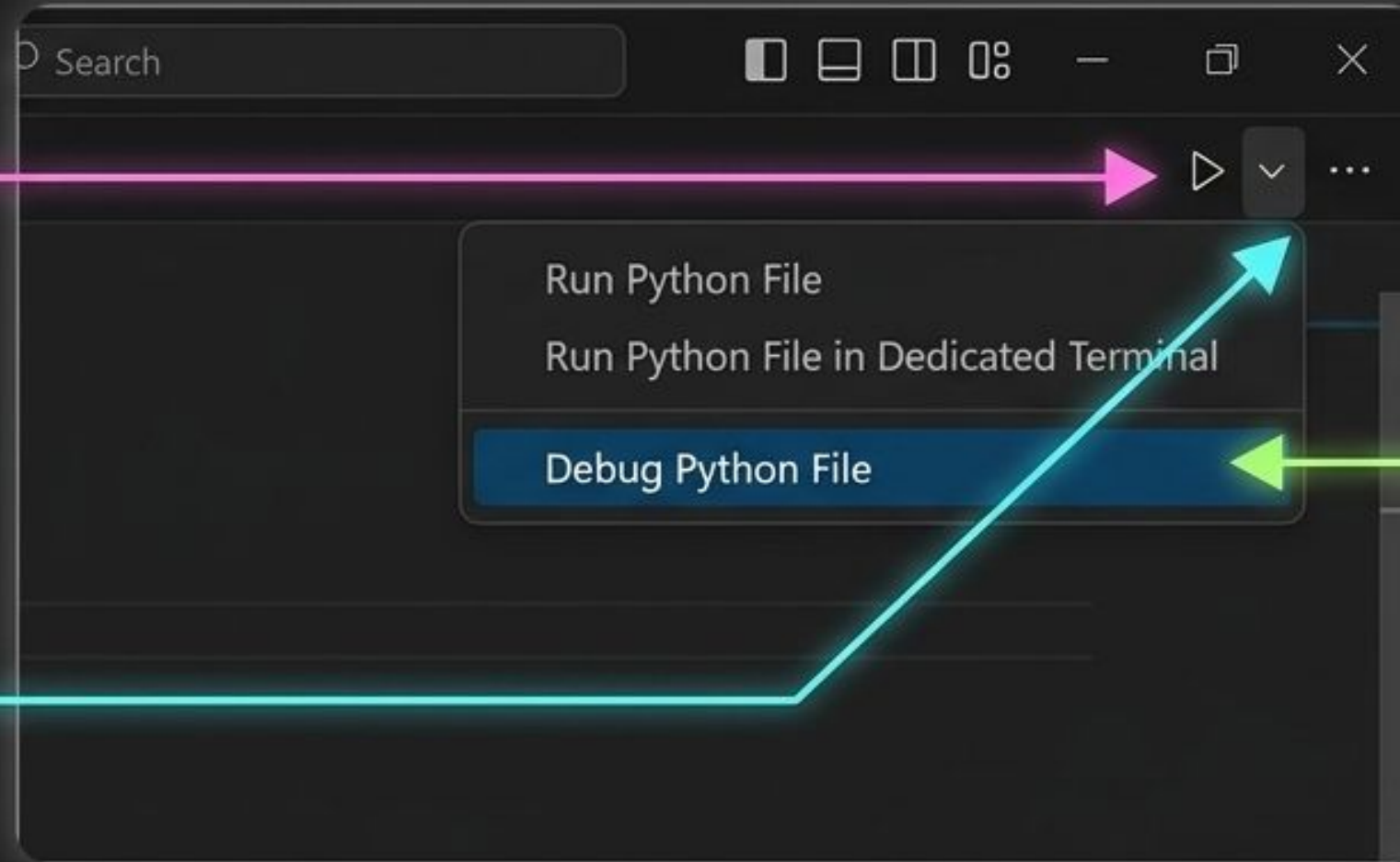
0 Break Point. Clique ao lado do número da linha para adicionar esta bola vermelha. É aqui que o nosso programa vai pausar no tempo.

Passo 4: Iniciando o Depurador

1. Localize o botão de Play no canto superior direito do VS Code.

2. Clique na seta para baixo ao lado do botão Play.

3. Selecione a opção **Debug Python File**.



O código começará a rodar, mas vai parar subitamente na linha do Break Point.
Não se assuste, é exatamente isso que queremos!

Passo 5: O Loop em Câmera Lenta

O Código (Estático)



Clique no botão Avançar (Step Over) para executar uma linha por vez.

```
2  
3 total = 0  
4 for valor in despesas:  
5     total = total + valor  
6
```

O Inspetor (A Memória em Tempo Real)

Volta 1

✓ VARIABLES

✓ Locals

- > Special variables
- ✓ Locals
 - valor: 15.50
 - total: 0



Volta 2

- > Special variables
- ✓ Locals
 - valor: 4.90
 - total: 15.50

O Inspetor permite acompanhar o valor das variáveis durante a execução. O loop deixa de ser abstrato e se torna perfeitamente visível.

Síntese: O Poder da Automação Financeira

```
print(f'O total de despesas é: R$ {total}')
```

```
> O total de despesas é: R$ 33.0
```

1. Armazenamento

Estruturamos os dados (listas).

2. Automação

Processamos em massa (for loops).

3. Inspeção

Garantimos a lógica (debugger).

4. Resultado

Dashboard SmartFinance atualizado.

Você acabou de construir o motor financeiro do SmartFinance.
O terminal não é mais uma caixa preta, mas a sua ferramenta de trabalho.

Desafio: Substitua os valores da lista de despesas pelos seus gastos reais desta semana e rode o depurador novamente!